BuddyBird

Speech training for parrots

An app that plays words to a parrot while its owner is away and records the bird's attempts to imitate them. Released on Google Play and the App Store.

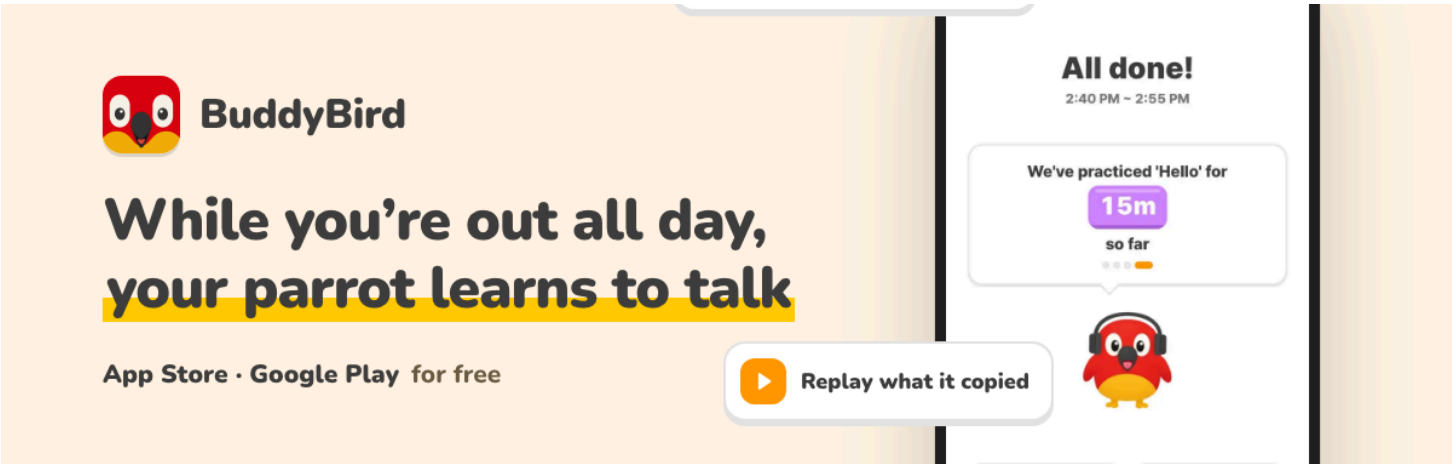
Period

Apr 2026 – Present

Role

Team lead, team of three · mobile, backend, deployment

About 1,400 **cumulative downloads and 30–40 daily active users as of Oct 2026**



What it does

- Repeated word playback and automatic recording of parrot speech
- Classification of parrot sounds against human speech and noise
- Remote WebRTC pet camera for monitoring training sessions
- Lock-screen controls, background playback, and custom word recordings

My contribution

- Segmented vocalizations with Silero VAD and integrated hardware AEC and WebRTC AEC3 on iOS and Android to remove playback echo.
- Built the WebRTC viewer and connected devices across networks with Cloudflare Realtime STUN/TURN.
- Integrated the parrot-sound classifier into FastAPI; moved deployment from Docker and ECS to an SQS and Lambda pipeline.
- Automated EAS and GitHub Actions builds, Maestro end-to-end tests, and mobile releases.

A problem I worked through

Problem

Words played by the app could re-enter the microphone and be mistaken for the parrot's voice.

How I solved it

I combined vocalization detection with native echo cancellation to separate playback from recorded speech.

Process and decisions

From the problem to an MVP

The midterm review framed the gap: video loops and generic pet cameras could not play an owner's voice or teach selected words. We chose voice-based word training as the first feature and launched the app on June 10, 2026.

Consented audio collection

In the initial MVP, VAD isolated intervals from training recordings collected with user consent. Audio went to S3 and metadata such as duration and format to RDS. By Aug 14, the pipeline had collected 11,250 clips totaling about 1.87 GB for model validation.

Evolving the shipped product

Remote pet monitoring and training results grew from the review roadmap into product features. To prevent playback audio from confusing vocalization detection, I integrated Silero VAD and platform-specific echo cancellation into the audio path.

What the midterm review validated

As presented in Aug 2026

The MVP launched on June 10 to test user demand and whether parrot vocalizations could be collected for later analysis.

824
Cumulative downloads by Aug 15

~32
Daily active users, Jul 1–Aug 15

11,250
Audio clips collected by Aug 14

Audio collection and classification

01 Record during training with user consent

02 Isolate sound intervals with VAD

03 Store audio in S3 and metadata in RDS

04 Classify parrot sounds using Perch 2.0 features and logistic regression

We selected Perch 2.0 after Leave-One-Bird-Out validation across conures and quakers. The early DTW subsequence prototype for word imitation reached 0.83 AUC and 0.77 balanced accuracy. These are research results from the midterm review.

Emergency detection and daily activity reports were presented as future work at that stage.

Technology

React Native (Expo) **WebRTC** **Python · FastAPI** **AWS ECS · SQS · Lambda** **Docker · Pulumi**

EAS · GitHub Actions · Maestro

Alledin

A personal allergy guide

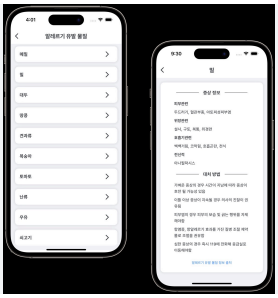
A mobile service that finds franchise restaurant menus compatible with a user's food allergies.

Period

May – Nov 2023

Role

Four-person team · one of two backend developers



What it does

- Save allergies from 21 designated allergens
- Explain symptoms, responses, and severity
- Search restaurants and menus using location and allergy data

My contribution

- Collected roughly 10,000 franchise locations, products, and allergen records, normalized them as CSV, and stored them in AWS RDS.
- Designed MVP wireframes in Figma and standardized server errors with an Exception Enum.
- Converted public-data coordinates for the map API and cleaned inconsistent records.

A problem I worked through

Problem

The service queried a user's allergies for each menu item, causing restaurant searches to take roughly one minute.

How I solved it

Changed the service to load the user's allergy data once and reuse it, reducing repeated database reads.

Process and decisions

Preparing the data

We gathered location, product, and allergen information from franchises with more than 100 branches. I removed unusable public records, normalized inconsistent names, prepared CSV files for RDS, and examined coordinate conversion for the map API.

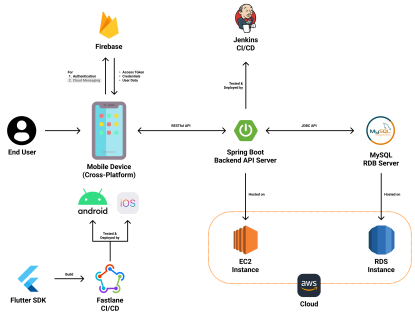
Fixing the search path

I traced the request through the controller, service, and database and found a repeated allergy query for every menu. Loading allergy data once and reusing it removed the bottleneck behind searches that took about a minute.

What remained

New and discontinued menu items change continuously. A production service would need a regular data refresh and validation workflow.

System architecture



Technology

Java 17 · Spring Boot 3 MySQL · AWS RDS AWS EC2 Figma

Distrip

Discover destinations through photos

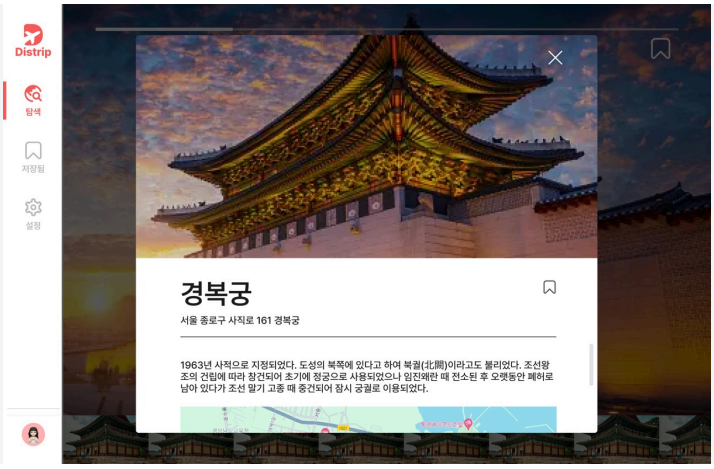
A web service where undecided travelers browse destination photos and save places they like.

Period

Jan – Apr 2024

Role

Three-person team · sole backend developer



What it does

- Random destination photos with details
- Saved destinations and bookmark view

My contribution

- Collected and cleaned roughly 1,000 tourism records from the Korea Tourism Organization and operated AWS RDS.
- Built APIs for randomized destinations and user bookmarks.

A problem I worked through

Problem

With a limited set of photos, returning visitors could keep seeing destinations they had already viewed.

How I solved it

Saved each signed-in user's last viewed index and resumed from the next item in a shuffled list.

Process and decisions

Photo-first discovery

The interface shows a destination photo first and reveals details after the user selects it. Places of interest can be saved and revisited through bookmarks.

Reducing repeat exposure

With roughly 1,000 records, returning visitors could see the same photos repeatedly. I kept a shuffled order and the last viewed index for signed-in users so browsing could resume at the next photo.

What remained

The original portfolio identified preference-based recommendations and sharing as later improvements once the data set and user base grew.

Technology

Java 21 · Spring Boot 3 **MySQL · AWS RDS** **Korea Tourism Organization data**

Lightweight CRM

Customer management for small teams

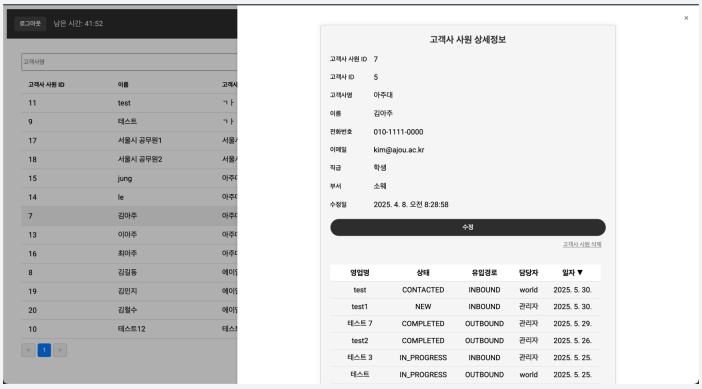
A web service for small sales teams to manage companies, contacts, sales history, and schedules in one place. Presented at Korea Computer Congress 2025.

Period

Feb – Jun 2025

Role

Solo project · planning, design, full-stack development



What it does

- Sales staff, companies, and contact management
- Sales and contact histories with linked detail views
- Email reminders for contact schedules

My contribution

- Selected essential sales workflows, designed the screens, and developed and deployed the frontend and backend.
- Added a slide-out detail panel connected to each selected list item.

A problem I worked through

Problem

Multiple users working with the same customer data needed distinct access permissions.

How I solved it

Assigned account roles to limit data access and prevented lower-privilege users from changing roles.

Process and decisions

Designed around core workflows

After reviewing Salesforce, HubSpot, and monday, I scoped the service to companies, contacts, sales history, and follow-up schedules. Selecting a list row opens a slide-out panel with linked details.

Schedules and permissions

A scheduled contact triggers email reminders. Account roles constrain which records a user can view and prevent lower-privilege users from changing roles.

Validation limit

The original presentation noted that the service was not adopted by a real company, so long-term usability remained untested. Field feedback would guide the next round of features.

System architecture

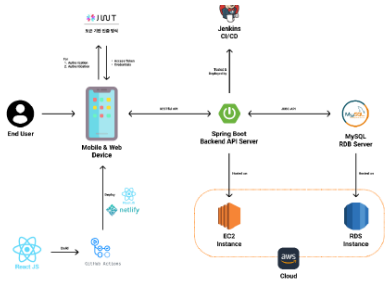


그림 4. System Architecture

Technology

React **Java 21 · Spring Boot 3** **JPA · MySQL** **AWS EC2 · RDS**